Review: Pushing remote branches

This reading contains the code used in the instructional videos from **Pushing remote branches**

Introduction

This follow-along reading is organized to match the content in the video that follows. It contains the same code shown in the next video. These code blocks will provide you with the opportunity to see how the code is written and can be used as a reference as you work through the course.

You can follow along in the reading as the instructor discusses the code or review the code after watching the video.

```
git checkout -b refactor
```

Code output:

Switched to a new branch 'refactor'

```
atom all_checks.py

(...)
def main():
    if check_reboot():
        print("Pending Reboot.")
        sys.exit(1)
    if check_disk_full(disk="/", min_gb=2, min_percent=10):
        print("Disk full.")
        sys.exit(1)
(...)


(...)
def check_root_full():
    """Returns True if the root partition is full, False otherwise."""
```

```
    return check_disk_full(disk="/", min_gb=2, min_percent=10)

def main():
    if check_reboot():
        print("Pending Reboot.")
        sys.exit(1)
    if check_root_full():
        print("Root partition full.")
        sys.exit(1)
(...)
```

```
./all_checks.py
```

Code output:

```
Everything ok.
```

```
git commit -a -m 'Create wrapper function for check_disk_full'
```

Code output:

```
[refactor e914aee] Create wrapper function for check_disk_full

 1 file changed, 8 insertions(+), 2 deletions(-)
```

```
(...)
def check_root_full():
    """Returns True if the root partition is full, False otherwise."""
    return check_disk_full(disk="/", min_gb=2, min_percent=10)

def main():
    checks = [
            (check_reboot, "Pending Reboot."),
            (check_root_full, "Root partition full"),
            ]

    for check, msg in checks:
        if check():
            print(msg)
            sys.exit(1)

    print("Everything ok.")
```

```
    sys.exit(0)
(...)
```

```
./all_checks.py
```

Code output:

Everything ok.

```
git commit -a -
m 'Iterate over a list of checks and messages to avoid code duplication'
```

Code output:

[refactor 75bdd43] Iterate over a list of checks and messages to avoid code duplication

1 file changed, 8 insertions(+), 6 deletions(-)

```
(...)
def main():
    checks = [
            (check_reboot, "Pending Reboot."),
            (check_root_full, "Root partition full"),
            ]
everything_ok = True
    for check, msg in checks:
        if check():
            print(msg)
            everything_ok = False

    if not everything_ok:
        sys.exit(1)

    print("Everything ok.")
   sys.exit(0)
(...)
```

```
./all_checks.py
```

Code output:

Everything ok.

```
git commit -a -m 'Allow printing more than one error message'
```

Code output:

[refactor cbee3f7] Allow printing more than one error message

1 file changed, 7 insertions(+), 1 deletion(-)

```
git push -u origin refactor
```

Code output:

Username for 'https://github.com': redquinoa

Password for 'https://redquinoa@github.com':

Enumerating objects: 11, done.

Counting objects: 100% (11/11), done.

Delta compression using up to 4 threads

Compressing objects: 100% (9/9), done.

Writing objects: 100% (9/9), 1.34 KiB | 1.34MiB/s, done.

Total 9 (delta 3), reused 0 (delta 0)

remote: Resolving deltas: 100% (3/3), completed with 1 local object.

remote:

remote: Create a pull request for 'refactor' on GitHub by visiting:

remote: https://github.com/redquinoa/health-checks/pull/new/refactor

remote:

To https://github.com/redquinoa/health-checks.git

 * [new branch]   refactor -> refactor

Branch 'refactor' set up to track remote branch 'refactor' from 'origin'.